

OCR Evaluation Framework

Standard Operating Procedure

Wartime Posters · Six-Tool Evaluation

Intended for

Librarians / Archive staff / Researchers with no programming background

No programming or OCR expertise is required.

Follow the steps to run and evaluate six OCR tools on the wartime poster material.

Contents

1. About This Manual.....	3
1.1 Overall workflow	3
1.2 Time required.....	3
1.3 What to prepare in advance	3
2. Installation (first time only)	4
2.1 Install Python.....	4
2.2 Install the evaluation dependencies	4
2.3 Set up the project folder.....	5
3. Obtaining the OCR Results.....	6
3.1 How the six tools differ	6
3.2 Google Cloud Vision (supplied results)	6
3.3 eScriptorium (web platform, manual export)	6
3.4 Transkribus (web platform, manual export).....	6
3.5 Running Docling, Surya and PaddleOCR locally	8
4. Preparing the Ground Truth Files.....	9
4.1 Layout ground truth (Label Studio JSON)	9
4.2 Text ground truth (Word documents).....	9
5. Running the Evaluation	10
5.1 Fill in the difficulty tiers (TIER_MAP)	10
5.2 Run the main evaluation	10
5.3 Run the three analysis scripts	10
6. Viewing the Results	12
7. Troubleshooting	13
Appendix A. Output Files	14
A.1 Main result files	14
A.2 Columns in summary_by_tool.csv.....	14

1. About This Manual

This manual is the operating guide for the KU Leuven Library wartime poster OCR evaluation framework. It is written for staff with no programming or OCR background. By following it you will be able to:

- Run six OCR tools over 40 (or more) wartime posters and evaluate their output
- Compute layout detection (IoU / F1), Language Region Recognition Rate (LRR), Character Error Rate (CER) and Coverage
- Produce CSV reports that open directly in Excel

1.1 Overall workflow

The work falls into three stages. Stage A is done once; each new batch of material starts at Stage B.

Stage	What you do	Section	Produces
A Install	Install Python and the evaluation dependencies; place the project folder	2 (first time only)	A working environment
B Recognise	Obtain each tool's OCR output for the posters	3	Files in adapters/*_output/
C Evaluate	Prepare ground truth, run the evaluation and analysis scripts	4-6	CSV reports in results/

Only three of the six tools (Docling, Surya, PaddleOCR) can be re-run on your own machine. Google Cloud Vision requires a paid account, and eScriptorium and Transkribus are web platforms whose output must be exported by hand.

1.2 Time required

Task	Estimated time	Notes
First-time installation and setup	30-60 min	Done once
Local recognition (3 tools x 40 posters)	1-4 hours	First run downloads models, several hundred MB each
Web platform recognition and export (2 tools)	1-2 hours	Includes upload, queueing and export
Organising ground truth and tool output	15-30 min	Depends on the number of posters
Running the evaluation and analysis scripts	10-30 min	Depends on machine speed

1.3 What to prepare in advance

- The layout ground truth JSON exported from Label Studio (Wartime_Posters_Layout_labelstudio_JSON.json)
- One text ground truth Word document per poster (named ground_truth_<poster_id>.docx)
- The poster images, named by poster ID (e.g. IE4984785.jpg) - required for local recognition
- The poster difficulty table (easy / medium / hard) - request this from the researcher
- A computer with an internet connection (needed for installation and model downloads)

Windows users: type the commands in this manual into Command Prompt or Windows PowerShell. Mac users: use Terminal. Where a command is shown as python, Mac users should type python3.

2. Installation (first time only)

2.1 Install Python

Windows

Step 1 Open a browser and go to <https://www.python.org/downloads/>

Step 2 Click the yellow button 'Download Python 3.x.x' (any version 3.9 or above)

Step 3 Run the downloaded installer

On the first installer screen you must tick 'Add Python to PATH'. If you miss it, every command below will fail.

Step 4 Click 'Install Now' and wait for it to finish

Step 5 Press Win+R, type cmd, press Enter, then verify with:

```
python --version
```

If it prints 'Python 3.x.x', the installation succeeded.

Mac

Step 1 Go to <https://www.python.org/downloads/> and download the macOS installer (.pkg)

Step 2 Double-click the installer and follow the prompts

Step 3 Open Terminal (search for it in Spotlight) and verify with:

```
python3 --version
```

2.2 Install the evaluation dependencies

Move into the project folder, then install everything with one command:

```
cd <path to the project folder>
```

```
pip install -r requirements.txt # Windows
```

```
pip3 install -r requirements.txt # Mac
```

This installs four required libraries at the versions the framework is pinned to:

Library	Version	Purpose
langdetect	1.0.9	Identifies the language of each region - the basis of the LRR metric
python-Levenshtein	0.27.3	Computes the Character Error Rate (CER)
python-docx	1.2.0	Reads the Word text ground truth
lxml	6.0.2	Reads PAGE XML (the eScriptorium / Transkribus export format)

langdetect is not optional. Without it, the language-dependent metrics (LRR, label accuracy, macro-F1 per label) cannot be computed from real language identification, and the only alternative is a fixed-column rule that holds for just 18 of the 40 posters. The program therefore refuses to start rather than produce a full results table built on the wrong rule.

If a dependency is missing, the program stops immediately and prints the exact command to run:

```
=====
REQUIRED DEPENDENCIES MISSING - refusing to run
=====
- langdetect==1.0.9

  pip install langdetect==1.0.9
=====
```

If Windows reports 'pip is not recognised', reinstall Python and make sure 'Add Python to PATH' is ticked.

2.3 Set up the project folder

Step 1 Copy the whole ocr-evaluation-framework folder supplied by the researcher to your Desktop or another easy-to-find location

Step 2 Check that it has the structure below. Folders marked with an arrow are ones you put files into.

Path	Description
ocr-evaluation-framework/	Project root
images/	<-- Poster images (IE4984785.jpg ...), required for local recognition
ground_truth/layout/	<-- The layout ground truth JSON from Label Studio
ground_truth/text/	<-- The text ground truth Word documents
adapters/google_output/	<-- Google Vision JSON results
adapters/escriptorium_output/	<-- PAGE XML exported from eScriptorium
adapters/transkribus_output/	<-- PAGE XML exported from Transkribus
adapters/docling_output/	Written automatically by recognition/run_docling.py
adapters/surya_output/	Written automatically by recognition/run_surya.py
adapters/paddle_output/	Written automatically by recognition/run_paddle.py
adapters/tool_*.py	Code that reads each tool's output - do not modify
recognition/	Local recognition scripts and their README (Section 3)
metrics/ alignment/ report/	Metric computation code - do not modify
analysis/	Additional analysis scripts (Section 5)
evaluate.py	The main evaluation script
requirements.txt	The evaluation dependency list
results/	Evaluation output is written here automatically

The tool output folders are under adapters/, not tools/. Files placed elsewhere will not be found.

3. Obtaining the OCR Results

The goal of this section is to fill all six *_output folders under adapters/ with the corresponding tool's results. If the researcher has already given you these files, skip to Section 4.

3.1 How the six tools differ

Tool	Can you re-run it?	How to obtain the results
Google Cloud Vision	No	Requires a paid GCP account. Use the JSON supplied by the researcher
eScriptorium	No	Web platform: upload, recognise, export PAGE XML by hand (3.3)
Transkribus	No	Web platform: upload, recognise, export PAGE XML by hand (3.4)
Docling	Yes	Run recognition/run_docling.py (3.5)
Surya	Yes	Run recognition/run_surya.py (3.5)
PaddleOCR	Yes	Run recognition/run_paddle.py (3.5)

The evaluation program does not perform recognition for you. evaluate.py only reads result files that already exist under adapters/; if they are missing it reports an error. You must complete this section first.

3.2 Google Cloud Vision (supplied results)

Step 1 Locate the Google Vision result JSON files

Step 2 Check that each is named <poster_id>.json, for example IE4984785.json

Step 3 Place them all in adapters/google_output/

For 40 posters you should have 40 JSON files.

3.3 eScriptorium (web platform, manual export)

3.3.1 Get an account

Step 1 Ask your institution's IT department for the eScriptorium instance address, or apply at <https://escriptorium.inria.fr>

Step 2 Confirm the verification email and log in

3.3.2 Upload and recognise

Step 1 Click 'Create new Project' and name it 'Wartime Posters'

Step 2 Open the project, click 'Create new Document', go to the 'Images' tab and 'Upload Images' to upload all the posters

Step 3 Go to the 'Transcribe' tab and select a pre-trained historical print model

Step 4 Click 'Transcribe' and wait for the progress bar to reach 100%

3.3.3 Export

Step 1 Click 'Export', choose the 'PAGE XML' format, and download the ZIP

Step 2 Unzip it and place the .xml files in adapters/escriptorium_output/

3.4 Transkribus (web platform, manual export)

Step 1 Go to <https://app.transkribus.org>, register and log in (the free allowance is usually enough for 40 posters)

Step 2 Click 'Collections' then 'New Collection' and name it 'Wartime Posters'

Step 3 Click 'Upload Document' and upload all the poster images

- Step 4** Select all pages, click 'Tools' then 'Text Recognition', and choose a print model (for example 'Transkribus Print M1')
- Step 5** Click 'Run' and watch progress on the 'Jobs' page
- Step 6** When it finishes, click 'Export', choose 'PAGE XML', download and unzip
- Step 7** Place the .xml files in adapters/transkribus_output/

3.5 Running Docling, Surya and PaddleOCR locally

These three tools must each be installed in their own virtual environment. Docling and Surya both depend on torch but pin incompatible versions, and PaddleOCR requires paddlepaddle. Installing all three into one environment is the most common cause of failure in this workflow. Each step below creates an environment and deactivates it when done.

Before you start: make sure the poster images are in images/ and named by poster ID:

```
images/  
IE4984785.jpg  
IE4984820.jpg  
...
```

3.5.1 Docling

```
cd <path to the project folder>  
  
python -m venv .venv-docling  
source .venv-docling/bin/activate      # Mac  
.venv-docling\Scripts\activate        # Windows  
  
pip install -r recognition/requirements-docling.txt  
  
python recognition/run_docling.py --poster IE4984785 # test one first  
python recognition/run_docling.py                  # then all of them  
  
deactivate
```

Results are written to adapters/docling_output/.

3.5.2 Surya

```
python -m venv .venv-surya  
source .venv-surya/bin/activate      # Mac  
.venv-surya\Scripts\activate        # Windows  
  
pip install -r recognition/requirements-surya.txt  
  
python recognition/run_surya.py  
  
deactivate
```

Results are written to adapters/surya_output/results.json. Surya writes all posters into a single file - this is normal and is its own format.

3.5.3 PaddleOCR

```
python -m venv .venv-paddle  
source .venv-paddle/bin/activate      # Mac  
.venv-paddle\Scripts\activate        # Windows  
  
pip install -r recognition/requirements-paddle.txt  
  
python recognition/run_paddle.py --poster IE4984785 # test one first  
python recognition/run_paddle.py                  # then all of them  
  
deactivate
```

Results are written to adapters/paddle_output/.

Each script also writes a run_metadata.json into its output folder, recording the tool version, settings, platform and timestamp. Do not delete these - they are the evidence that the results can be reproduced.

All three scripts accept --skip-existing. If a run is interrupted, add that flag and re-run: it skips posters already done and continues from where it stopped. The first run of each tool downloads its models (several hundred MB), so stay connected.

4. Preparing the Ground Truth Files

4.1 Layout ground truth (Label Studio JSON)

Step 1 Open Label Studio, log in, and open the wartime poster annotation project

Step 2 Click 'Export' in the top right, choose the 'JSON' format, and download

Step 3 Rename the file to:

Wartime_Posters_Layout_labelstudio_JSON.json

Step 4 Place it in ground_truth/layout/

4.2 Text ground truth (Word documents)

Step 1 Check that each poster has one Word document named ground_truth_<poster_id>.docx (for example ground_truth_IE4984785.docx)

Step 2 Place them all in ground_truth/text/

The text ground truth uses bracketed tags such as [Left Column: German], [Header] and [Body] to separate content. The evaluation program recognises and removes these tag lines automatically.

5. Running the Evaluation

5.1 Fill in the difficulty tiers (TIER_MAP)

The evaluation needs to know each poster's difficulty tier so it can report results grouped by complexity. This has to be entered into the program by hand.

Step 1 Open evaluate.py in the project root with a text editor

Windows: right-click the file, choose 'Open with' then 'Notepad'. Mac: right-click, 'Open With' then 'TextEdit'.

Step 2 Find the TIER_MAP block and fill in each poster's tier:

```
TIER_MAP: dict[str, str] = {
    "IE4984785": "easy",
    "IE4984820": "medium",
    "IE4985178": "hard",
    # ... 40 posters in total
}
```

Step 3 Save the file (Ctrl+S / Command+S)

For this study the tiers are: 6 easy, 10 medium, 24 hard - 40 posters in total.

5.2 Run the main evaluation

Return to your main environment (not the .venv-* environments from Section 3):

```
cd <path to the project folder>

python evaluate.py      # Windows
python3 evaluate.py    # Mac
```

On startup the program checks its dependencies and prints their versions, then shows progress poster by poster:

```
Dependencies : langdetect==1.0.9, python-Levenshtein==0.27.3, ...
Tools       : google, escriptorium, transkribus, surya, docling, paddle
IoU threshold : 0.5 (tau)

--- Tool: google ---
IE4984785 (68 GT regions)... P=0.50 R=0.09 F1=0.15 LRR=0.00 CER=0.009
```

Useful options:

Command	What it does
python evaluate.py	Evaluate all tools on all posters (the usual case)
python evaluate.py --tools google surya	Evaluate only the named tools
python evaluate.py --posters IE4984785 IE4984820	Evaluate only the named posters (useful for testing)
python evaluate.py --iou 0.3	Use a different IoU threshold (default 0.5)

5.3 Run the three analysis scripts

Once the main evaluation has finished, run these three scripts in the same environment. They do not run automatically.

```
python analysis/tau_sensitivity.py    # IoU threshold sensitivity
python analysis/coverage_analysis.py  # recognition error vs omission
python analysis/font_analysis.py      # typesetting vs CER
```

Script	Question it answers	Output files
tau_sensitivity.py	Does the tool ranking change if the IoU threshold changes?	tau_sensitivity.csv tau_sensitivity_delta.csv
coverage_analysis.py	Is the tool reading the text wrongly, or not reading it at all?	coverage_analysis.csv coverage_summary.csv
font_analysis.py	Are wood-type display headers harder to recognise than machine-set body text?	font_analysis_by_region.csv font_analysis_variance.csv

6. Viewing the Results

Step 1 Open the results/ folder inside the project

Step 2 Double-click any CSV file to open it in Excel (or Numbers, or LibreOffice Calc)

summary_by_tool.csv is the one you will use most: one row per tool, summarising every average metric, suited to cross-tool comparison.

results/run_metadata.json records the dependency versions, platform and timestamp of the run. It is not a result, but please archive it with them.

7. Troubleshooting

Message or symptom	Cause and fix
REQUIRED DEPENDENCIES MISSING - refusing to run	Not a fault. The program refuses to run when a dependency is missing rather than return wrong numbers. Run the pip install command it prints.
FileNotFoundError: Layout GT not found	The JSON in ground_truth/layout/ has the wrong name. It must be Wartime_Posters_Layout_labelstudio_JSON.json, including capitalisation.
Google Vision output not found: .../IE4984785.json	That poster's result file is missing from adapters/google_output/, or is not named by poster ID.
Surya / Docling / Paddle result files do not exist	Section 3 has not been done. The evaluation does not perform recognition; run the scripts in recognition/ first.
surya_ocr: command not found	Surya is not installed, or the .venv-surya environment is not active. Activate it, then pip install -r recognition/requirements-surya.txt
ValueError: Unknown argument (PaddleOCR)	PaddleOCR 2.x is installed. run_paddle.py supports 3.x only. Install using requirements-paddle.txt.
image not found (recognition scripts)	A filename in images/ is not exactly the poster ID (e.g. IE4984785_scan.jpg). Rename it.
torch version conflicts / installation errors	Two or more local tools were installed into one environment. Each needs its own venv - see 3.5.
No module named docx	Dependencies are not fully installed. Run pip install -r requirements.txt
pip is not recognised (Windows)	'Add Python to PATH' was not ticked during installation. Reinstall Python.
LRR is 0 everywhere, or clearly too low	Check that langdetect is installed and that the startup line shows langdetect==1.0.9.

Appendix A. Output Files

A.1 Main result files

File	Contents
summary_by_tool.csv	Macro-averaged metrics, one row per tool - the main reporting table
results_<tool>.csv	Per-poster detail for a single tool
results_all_tools.csv	All tools combined, suited to cross-tool comparison in Excel
summary_by_tier.csv	Results grouped by difficulty tier (easy / medium / hard)
summary_by_language.csv	CER grouped by language (de / nl / fr / xx)
f1_per_label.csv	F1 for each region class
tau_sensitivity_delta.csv	Whether the ranking changes with the IoU threshold
coverage_summary.csv	Breakdown of recognition error versus omission
font_analysis_variance.csv	Paired Header vs Body CER comparison
run_metadata.json	Dependency versions, platform and timestamp (not a result; for archiving)

A.2 Columns in summary_by_tool.csv

Column	Range	Meaning
tool	text	Tool name
layout_precision	0-1	Layout precision: of the regions the tool found, the proportion that are correct
layout_recall	0-1	Layout recall: of the ground truth regions, the proportion the tool found
layout_f1	0-1	Harmonic mean of precision and recall; higher is better
layout_mean_iou	0-1	Mean overlap of the matched regions
lrr	0-1	Language Region Recognition Rate: of the language-bearing regions, the proportion located correctly AND assigned the right language
label_acc	0-1	Accuracy of the region class assignment
cer	0 upwards	Character Error Rate; lower is better. Computed on the full page text
coverage	0 upwards	Characters output divided by ground truth characters. Around 1.0 is normal; well below 1 means text was missed